

3



Project Communication

Two electrical boxes for a rocket, manufactured by different contractors, were connected by a pair of wires. Thanks to a thorough preflight check, it was discovered that the wires had been reversed. After the rocket crashed, the inquiry board revealed that the contractors had indeed corrected the reversed wires as instructed.

In fact, both of them had.

Software engineering is a collaborative activity. The development of software brings together participants from different backgrounds, such as domain experts, analysts, designers, programmers, managers, technical writers, graphic designers, and users. No single participant can understand or control all aspects of the system under development, and thus, all participants depend on others to accomplish their work. Moreover, any change in the system or the application domain requires participants to update their understanding of the system. These dependencies make it critical to share information in an accurate and timely manner.

Communication can take many forms depending on the type of activity it is supporting. Participants communicate their status during weekly or bimonthly meetings and record it into meeting minutes. Participants communicate project status to the client during client reviews. The communication of requirements and design alternatives is supported by models and their corresponding documents. Crises and misunderstandings are handled through spontaneous information exchanges such as telephone calls, E-mail messages, hallway conversations, or ad hoc meetings.

In this chapter, we first motivate the need and importance of communication in software engineering. Next, we describe different aspects of project communication and their relationship with different tasks. Next, we survey a number of tools for supporting project communication. Finally, we describe an example of communication infrastructure.

3.1 Introduction: A rocket example

When realizing a system, developers focus on constructing a system that behaves according to specification. When interacting with other project participants, developers focus on communicating information accurately and efficiently. Even if communication may not appear to be a creative or challenging activity, it contributes as much to the success of the project as a good design or efficient implementation, as illustrated by the following example [Lions, 1996].

Ariane 501

June 4, 1996, 30 seconds into lift-off, Ariane 501, the first prototype of the Ariane 5 series, exploded. The main navigational computer experienced an arithmetic overflow, shut down, and handed control over to its twin backup, as it was designed to do. The backup computer, having experienced the same exception a few hundredths of a second earlier, had already shut down. The rocket, without a navigation system, took a fatal sharp turn to correct a deviation that had not occurred.

An independent board of inquiry took less than 2 months to document how a software error resulted into the massive failure. The navigational system of the Ariane 5 design was one of the few components of Ariane 4 that was reused. It had been flight tested and had not yet failed for Ariane 4.

The navigation system is responsible for calculating course corrections from a specified trajectory based on input from the inertial reference system. An inertial reference system allows a moving vehicle (e.g., a rocket) to compute its position solely based on sensor data from accelerometers and gyroscopes, that is, without reference to the outside world. The inertial system must first be initialized with the starting coordinates and align its axis with the initial orientation of the rocket. The alignment calculations are done by the navigation system before launch and need to be continuously updated to take into account the rotation of the Earth. Alignment calculations are complex and take approximately 45 minutes to complete. Once the rocket is launched, the alignment data are transferred to the flight navigational system. By design, the alignment calculations continue for another 50 seconds after the transfer of data to the navigation system. The decision enables the countdown to be stopped after the transfer of alignment data takes place but before the engines are ignited without having to restart the alignment calculations (that is, without having to restart a 45-minute calculation cycle). In the event the launch succeeds, the alignment module just generates unused data for another 40 seconds after lift-off.

The computer system of Ariane 5 differed from Ariane 4. The electronics was doubled: two inertial reference systems to compute the position of the rocket, two computers to compare the planned trajectory with the actual trajectory, and two sets of control electronics to steer the rocket. If any component would fail, the backup system would take over.

The alignment system, designed for onground calculations only, used 16-bit words to store horizontal velocity (more than enough for displacements due to the wind and to the rotation of the earth). Thirty seconds into flight, the horizontal velocity of Ariane 5 caused an overflow, raised an exception that was handled by shutting down the onboard computer and handing control to the backup system.

Discussion

There had not been adequate test coverage of the alignment software. It had been subjected to thousands of tests, but not one of them included an actual trajectory. The navigation system was tested individually. Tests were specified by the system team and executed by the builders of the navigation system. The system team did not realize that the alignment module could cause the main processor to shut down, especially not in flight. There had been a failure to communicate between the component team and the system team.

In this chapter, we discuss project communication within a software development project. This topic is not specific to software engineering. Communication is, however, pervasive throughout a software development project. The cost of communication failure can have a high, and sometimes fatal, impact on the project and the quality of the delivered system.

3.2 An overview of project communication

Communication needs and tools that support communication are numerous and diverse. To facilitate our discussion, we introduce the following classification and definitions (Figure 3-1):

- A **communication mode** refers to a type of information exchange that has defined objectives and scope. A client review is an example of a communication mode during which the client reviews the content and quality of a project deliverable. Problem reporting is another example of a communication mode during which a user reports a bug to the developers. A communication mode can be **scheduled** if it is a planned event or **event-driven** if it occurs nondeterministically. Client reviews are usually scheduled, whereas problem reports are event driven.
- A **communication mechanism** refers to a tool or procedure that can be used to transmit and receive information and support a communication mode. Smoke signals and fax machines are communication mechanisms. Communication mechanisms are **synchronous** if they require both sender and receivers to be available at the same time. Otherwise, communication mechanisms are called **asynchronous**. Smoke signals are synchronous, whereas fax machines are asynchronous.

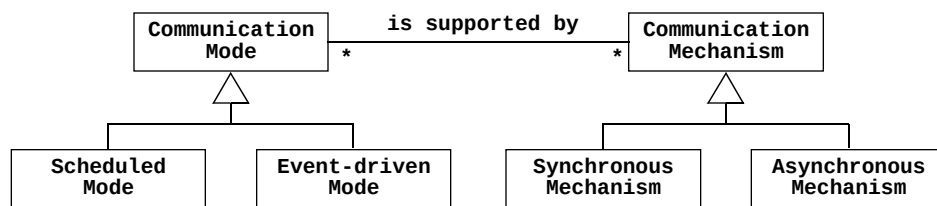


Figure 3-1 Classification of communication modes and mechanisms (UML class diagram).

Both synchronous and asynchronous communication mechanisms can be used to support a scheduled mode of communication. For example, in Figure 3-2, either smoke signals or a fax machine can be used for a client review. On the other hand, only asynchronous communication mechanisms can be used for supporting event-driven modes (reporting a problem with smoke signals may lead to loss of information if nobody was scheduled to watch the smoke. Note that a communication mode can be supported by many different communication mechanisms: The requirements analysis document can be faxed to the client, and the client sends back her

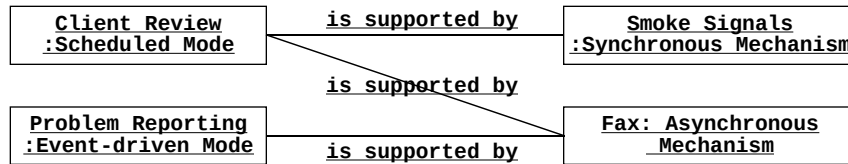


Figure e 3-2 Examples of modes and mechanisms (UML class diagram). Both scheduled modes and event-driven modes can be supported by asynchronous mechanisms. Event-driven modes, however, can only be supported by asynchronous mechanisms.

comments using smoke signals. Similarly, the same mechanism can support many communication modes: The fax machine can receive either problem reports or comments from a client review.

In Section 3.3, we review the different communication modes that communication can be used in a project. The scheduled modes reviewed include client reviews, status reports, and system releases. Event-driven modes include requests for changes, clarifications, and issue resolution. In Section 3.4, we survey a number of communication mechanisms available to project participants. In particular, we focus on meetings, reviews, report forms, questionnaires, electronic mail, newsgroups, and groupware. In Section 3.5, we examine a specific combination of communication modes and mechanisms in an example project.

3.3 Modes of communication

Scheduled communication occurs according to a periodic process (Table 3-1). For example, a team reviews its status every week, a client reviews the project status every month. These are focused, well-organized, and usually face-to-face points of communication directed at discovering and addressing a broad range of issues.

Event-driven communication is caused by a specific event (Table 3-2), such as the discovery of an incompatibility between two subsystems, the need for a new feature, or the necessity for resolving different interpretations before a formal review. These are spontaneous, ad hoc, and possibly asynchronous points of communication directed at resolving a single issue quickly.

Software projects require support for periodic and event-driven communication. Not all communication needs can be addressed through scheduled events and formal processes, whereas exchanging information only when needed or required results in additional crises due to miscommunications and omissions. In this section, we describe both types of modes.

Table 3-1 Scheduled modes of communication

Mode	Objectives	Scope
Problem definition (Section 3.3.1)	• Extract requirements knowledge from client and user • Extract domain knowledge from client and user	• Application domain • Functional and nonfunctional requirements • Delivery schedule
Client reviews (Section 3.3.2)	• Ensure that the system under construction is the one the client wants • Review feasibility of nonfunctional requirements (e.g, platform, performance) • Review the delivery schedule	• Functional and nonfunctional requirements • Delivery schedule
Project reviews (Section 3.3.3)	• Ensure that dependent subsystems can communicate • Anticipate late completion of critical subsystems • Disseminate operational knowledge across teams	• System design • Object design • Tests
Inspections/ Walkthroughs (Section 3.3.4)	• Ensure correctness of subsystem implementation • Improve quality of subsystem implementation • Disseminate operational knowledge across participants.	• Implementation
Status reviews (Section 3.3.5)	• Ensure adherence to the task plan • Ensure timely completion of planned deliverables • Dissemination of potential issues	• Tasks • Action items • Issues
Brainstorming (Section 3.3.6)	• Generate and evaluate solutions	• A single issue
Releases (Section 3.3.7)	• Dissemination of large volume of information (e.g., document, subsystem code)	• Project organization • Task plan • System, documents, and models
Postmortem review (Section 3.3.8)	• Capture and organization of lessons learned	• Project organization • Task plan • System and models

Table 3-2 Event-driven modes of communication

Mode	Objectives	Scope
Requests for clarification (Section 3.3.9)	• Clarification • Report an ambiguity in documentation or code	• Any
Requests for change (Section 3.3.10)	• Report a problem or a possible improvement	• Any
Issue resolution (Section 3.3.11)	• Reaching consensus • Formalizing chosen solution	• A single issue

3.3.1 Problem definition

The goal of **problem definition** is for the project manager and the client to agree on the scope of the system under construction. The project definition activity produces a problem definition document that outlines the domain and the functionality of the system. It also contains nonfunctional requirements such as platform specification or speed constraints. Consider the following example [FRIEND, 1994].

FRIEND Problem Statement

Weaknesses of today's emergency handling. Most local government emergency handling systems are established for normal call loads and are immediately overloaded when an unusual event or multiple events occur. During an emergency, responders may acquire useful information for the decision makers. This information is not transmitted by radio, unless asked for, due to limited air time.

Government services possess a wealth of information that is invaluable during emergencies. Fire, police, emergency medical services, public works, and building inspection departments compile large amounts of paper-based information. During an emergency, this paper-based information is extremely hard to use and is rarely used.

Crucial information is not always immediately available during emergencies. Incident location information, such as hazardous materials, gas line locations, etc., are not available or do not exist. There is a need for a system that would speed up response time to emergencies and provide information to make better decisions.

Functional requirements. The objective of the FRIEND project is to build a system that handles multiple emergency incidents at one time under unusual call loads. The system must be prepared for a worst case scenario of an emergency. The system must allow quick and remote response between police officers, fire fighters, maintenance workers, and the headquarters' dispatcher. A wide bandwidth will be used, so that the need for acknowledgment before transmission will not be a constraint.

The system would provide instant use of information, such as:

- geographic information—such as city maps, including above and underground utilities
- hazardous materials information (Hazmat)
- building specific information—such as gas and water lines, fire hydrants location, floor maps, etc.
- available government resources—such as the Emergency Operations Plan (EOP)

Following guidelines and the Emergency Operations Plan, the system will automatically notify personnel in the appropriate department, create task lists to follow, and allocate resources, as well as any other tasks that would save time during an emergency.

Every command post vehicle interacting with the FRIEND system will have a mobile computer that uses wireless communication to report to the headquarter dispatcher. The goal is to replace the first responders reporting input mechanism by an easy input and responsive user interface, voice recognition, a touch screen, or a pen-based system. All transactions on the system must be archived for future analysis.

Nonfunctional requirements. The FRIEND system will first be used by the Bellevue Police Department as a standalone product. Also, FRIEND is to be later used by a group of municipalities, with the ability to provide higher levels of government with local information. The hardware used by field service personnel will be exposed to weather and rough duty. FRIEND should be portable to existing hardware available in local governments. The prototype is expected to be scalable. System design should, therefore, be engineered to be expandable to deal with accidents on the state and federal levels.

Acceptance Criteria. The client considers this problem statement to be a broad definition and does not expect all functionality to be delivered in the first release. During the Requirement Engineering phase of the project, the client will negotiate with the software engineers an acceptable prototype for delivery. After the negotiation phase, the specific requirements for the client acceptance test will be frozen. The client expects to sign off on the negotiated deliverable within 4–6 weeks of the client presentation. As a minimum, the client expects the delivered prototype to be expandable in a future course. As a minimum acceptable test, the client expects the negotiated prototype to be successfully demonstrated on the Andrew system with a wireless component. As a desired acceptable test, the client expects the prototype to be successfully demonstrated in a field trial at the Bellevue Police Department.

The problem definition does not produce a complete specification of the statement. It is merely a preliminary requirements activity that establishes common ground between the client and the project. We discuss requirements activities in Chapter 4, *Requirements Elicitation* and Chapter 5, *Analysis*.

3.3.2 Client reviews

The goal of **client reviews** is for the client to assess the progress of the development and for the developers to confirm or change the requirements of the system. The client review can be used to manage expectations on both sides and to increase the shared understanding among participants. The focus of the review is on what the system does and what constraints are relevant to the client (e.g., performance, platform). In most cases, the review should not focus on the design or implementation of the system unless they impact the client or the user. Exceptions include software contracts that impose constraints on the development process, such as software system with safety requirements.

A client review is usually conducted as a formal presentation during which the developers focus on specific functionality with client. The review is preceded by the release of a work product, such as a specification document, an interface mock-up, or an evaluation prototype. At the outcome of the review, the client provides feedback to the developers. This feedback may

OWL Client acceptance test agenda**Date** : 12/5/1997**Time** : 3- 4:30pm**Location** : Forest Hall**Goal** : review of the system by the client and identification of open issues**Overview** :

- problem statement
- design goals
- system architecture
- Demo 1: Remote user interface and control
- Demo 2: Site editor
- Demo 3: 3D Visualization and speech user interface
- questions and answers
- review wrap up

Figure 3-3 An example of an agenda for a client review.

consist of a general approval or a request of detailed changes in functionality or schedule. Figure 3-3 depicts an example of an agenda for a client review.

3.3.3 Project reviews

The goal of a **project review** is for the project manager to assess status and for teams to review subsystem interfaces. Project reviews can also encourage the exchange of operational knowledge across teams, such as common problems encountered with tools or the system. The focus of the review depends on the deliverable under review. For system design, the decomposition and high-level subsystem interfaces are reviewed. For object design, the object interfaces are reviewed. For integration and test, the tests and their results are reviewed. The focus of project reviews should not be on functionality unless it impacts the feasibility or the status of the system.

A project review is typically conducted as a formal presentation during which each team presents its subsystem to the management or to teams that depend on the subsystem. The review is usually preceded by the release of a document (e.g, system design document) describing the aspects of the system under review (e.g., the subsystems interfaces). At the outcome of the review, the developers may negotiate changes in the interfaces and changes in schedule.

3.3.4 Inspections and walkthroughs

The objective of code **inspections** and **walkthroughs** is to increase the quality of a subsystem through peer review. In the case of a walkthrough, a developer presents to the other members of her team line-by-line the code she has written. The other team members challenge any suspicious code and attempt to discover as many errors as possible. The role of the

developer is to facilitate the presentation and answer the team's questions. In the case of an inspection, the members of the team focus on the compliance of the code with a predefined list of criteria (e.g., Does the code implement the specified algorithm? Does the code correctly use dependent subsystem interfaces?). In the case of inspections, the team leads the discussion, and the developer answers questions. The focus of the inspection or walkthrough is on the code, not on the programmer or the design.

Communication among participants is code based. The actual code is used as a common frame of reference. Inspections are similar to project reviews in their objective to increase quality and disseminate operational information. They differ from reviews in their formality, their limited audience, and their extended duration. Inspections and walkthroughs are widely used and have been effective at detecting defects early [Fagan, 1976], [Seaman & Basili, 1997]. We describe walkthroughs in Chapter 9, *Testing*.

3.3.5 Status reviews

Unlike client and project reviews that focus on the system, **status reviews** focus on tasks. Status reviews are primarily conducted in a team (e.g., weekly) and occasionally conducted in a project (e.g., monthly). The objective of status reviews is to detect deviations from the task plan and to correct them. Status reviews also encourages developers to achieve closure on pending tasks. The review of task status encourages the discussion of open issues and unanticipated problems, and, thus, encourage informal communication among team members. Often, solutions to common issues can be shared more effectively and operational knowledge disseminated when discussed within the scope of a team (as opposed to within the scope of the project).

Status reviews represent an investment in person power. Increasing the effectiveness of reviews has a global impact on the team performance. Status meetings should have an agenda available prior to the meeting describing the tasks and issues to be reviewed. This enables meeting participants to prepare for the meeting and redirect the agenda if an urgent issue needs to be discussed. Minutes for each meeting should be taken by a designated participant in order to capture as much information (mainly status and decisions) as possible. Minutes are made available to the participants for review as early as possible after the meeting. This encourages the minute taker to complete the minutes and for team members who missed the meeting to catch up with team status. Meeting minutes are subsequently referenced when related tasks are discussed or when clarification is needed. Moreover, meeting minutes represent a portion of the project history that can be analyzed after the project is completed.

3.3.6 Brainstorming

The goal of **brainstorming** is to generate and evaluate a large number of solutions for a problem. Brainstorming is usually done in face-to-face meetings but can also be done via E-mail or groupware. The fundamental idea behind brainstorming is that solutions proposed by one participant, however invalid, can trigger additional ideas and proposals from other participants. Similarly, the evaluation of proposals within a group leads to more explicit evaluation criteria

and to a more-consistent evaluation. In situations where no clear solutions exist, the brainstorming process can be facilitated by separating the generation and evaluation. Brainstorming has also the side effect of building consensus for the chosen solution.

3.3.7 Releases

The goal of a **release** is to make available a document or a subsystem to other project participants, often replacing an older version of the artifact. A release can be as simple as a two-line electronic message (see Figure 3-4) or can consist of several pieces of information: the new version of the artifact, a list of changes made since the last release of the artifact, a list of problems or issues yet to be addressed, and an author.

From: Al
Newsgroups: cs413.f96.architecture.discuss
Subject: SDD
Date: Thu, 25 Nov 03:39:12 -0500
Lines: 6
Message-ID: <3299B30.3507@andrew.cmu.edu>
MimeVersion: 1.0
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

An updated version of the API document for the Notification Subsystem can be found here: <http://decaf/~al/FRIEND/notifapi.html>

--Al
Notification Group Leader

Figure 3-4 An example of a release announcement.

Releases are used to make a large amount of information available in a controlled manner, by batching, documenting, and reviewing many changes together. Project and client reviews are typically preceded by a release of one or more deliverables.

The management of versions of documents, models, and subsystems is described in Chapter 10, *Software Configuration Management*.

3.3.8 Postmortem review

Postmortem reviews focus on extracting lessons from the development once the software is delivered. Postmortem reviews need to be conducted shortly after the end of the project so that minimal information is lost or distorted by subsequent experience. The end of the project is usually a good time to assess which techniques, methods, and tools have worked and have been critical to the success (or failure) of the system.

A postmortem can be conducted as a brainstorming session, a structured questionnaire followed by interviews, or individual reports written by teams (or participants). In all cases, the

<i>Question about problems that occurred</i>	What kinds of communication and negotiation problems have emerged in the development of the system?
<i>Question eliciting possible solutions to those problems</i>	Speculate on what kind of information structure is needed for team-based design in conjunction with a model-based object-oriented software engineering methodology. Do you feel the forums provided (Discuss, Issues, Documents, Announce, etc.) solved this challenge? Identify issues with the information structure and propose solutions.
<i>Question eliciting other aspects of the project that were either perceived as positive or could be improved</i>	What observations and comments do you have about the project concerning: • your expectations at the beginning of the project and how they evolved • the goals of this project • the use of use cases • the life cycle used in the project • the project management (meetings, communication, etc.) • the documentation process
<i>Open-ended catch-all question</i>	In addition to the above questions, please feel free to discuss any other issues and proposed solutions that you feel are relevant.

Figure e 3-5 An example of questions for a postmortem review.

areas covered should include the tools, methods, organization, and procedures used by the project. Figure 3-5 is an example of questions that can be used during a postmortem.

Even if the results of post mortems are not disseminated through the organization via formal channels (e.g., technical reports), they can still be disseminated indirectly through the project participants. Project participants are frequently reassigned to different projects or functions and often disseminate the lessons learned from the old project to other parts of the organization.

3.3.9 Request for clarification

Requests for clarification represent the bulk of the communication among developers, clients, and users. Requests for clarification are event driven. A participant may request clarification about any aspect of the system that may be ambiguous. Requests for clarification may occur during informal meetings, phone calls, E-mail, a form, or most other communication mechanism available to the project. Situations in which most information needs are handled through this mode are symptoms of a defective communication infrastructure. Such projects often face serious failures downstream resulting from misunderstandings and missing and misplaced information. Figure 3-6 depicts an example of request for clarification.

From: Alice
Newsgroups: cs413.architecture.discuss
Subject: SDD
Date: Thu, 10 Oct 23:12:48 -0400
Message-ID: <325DBB30.4380@andrew.cmu.edu>
MimeVersion: 1.0
Content-Type: text/plain; charset=us-ascii

When exactly would you like the System Design Document? There is some confusion over the actual deadline: the schedule claims it to be October 22, while the template says we have until November 7.

Thanks,
Alice

Figure 3-6 An example of a request for clarification.

3.3.10 Request for change

Request for change is an event-driven mode of communication. A participant reports a problem and, in some cases, proposes solutions. The participant reports a problem with the system itself, its documentation, the development process, or the project organization. Requests for change are often formalized when the number of participants and the system size is substantial. Change requests contain a classification (e.g., severe defect, feature request, comment), a description of the problem, a description of the context in which it occurs, and any amount of supporting material. Change request forms have been popularized by defects-tracking software. They can be applied to other aspects of the project (e.g., task plan, development process, testing procedures). Figure 3-7 depicts an example of change request form.

3.3.11 Issue resolution

Once problems have been reported and solutions proposed and evaluated, a single solution needs to be selected, communicated, and implemented. A flat organization may select a solution as part of the brainstorming process. A hierarchical organization or a crisis situation may require a single individual to select and impose a solution. In all cases, the decision needs to be documented and communicated to the relevant participants. Documentation of the resolution allows participants to refer back to the decision later in the project, in case of a misunderstanding. Effective communication of the decision enables participants to remain synchronized.

An issue base can serve as a communication mechanism for supporting problem tracking, brainstorming, and **issue resolution**. The issue base displayed in Figure 3-8 displays a list of messages exchanged as a result of issue resolutions. The message captions preceded by I: denote issues, those preceded by P: (for proposal) suggested solutions, A+ and A- denote arguments supporting and arguing against a solution. Finally, once issues are resolved, a single message, called a resolution, is posted to document the decision that was taken on the issue. We describe issue bases and issue modeling in Chapter 8, *Rationale Management*.

Modes of communication

75

Header information for identifying the change

Report number: 1291

Date: 5/3

Author: Dave

Synopsis: The FRIEND client crashes when empty forms are submitted.

Context information for locating the problem

Subsystem: User interface

Version: 3.4.1

Classification:

- missing/incorrect functionality
- convention violation
- **bug**
- documentation error

Severity:

- **severe**
- moderate
- annoying

Description of the problem and the rationale for change

Description:

Rationale:

Description of desired change

Proposed solution:

Figure e 3-7 An example of a change request form.**Figure e 3-8** An example of an issue base (Domino Lotus Notes database).

3.3.12 Discussion

In this section, we presented a broad range of modes of communication. Not all projects put the same emphasis on all modes of communication. Projects producing shrink-wrapped software may not have client reviews but may have sophisticated change report procedures. Other projects put more emphasis on testing than on code inspections and walkthroughs.

The modes of communication used in a project are usually dictated by the application domain of the system and the activities of the project. These modes are often made explicit and formalized into procedures to address critical communication paths. For example, project reviews make it easier for all participants to synchronize their state and confront issues that pertain to more than one or two subsystems. Project reviews become especially critical when the size of the project increases to the point that few participants have a global view of the system.

3.4 Mechanisms of communication

In this section, we describe several communication mechanisms that support information exchanges. We classify communication mechanisms in two broad categories. **Synchronous mechanisms**, such as telephone or video conference, require sender and receiver to be available at the same time, whereas **asynchronous mechanisms**, such as E-mail or fax, do not. We further classify synchronous mechanisms into same-place and different-place mechanisms. Same-place mechanisms (e.g., a hallway conversation) require the sender and receiver to be in the same location (e.g., a hallway), whereas different-place mechanisms do not.

Informal face-to-face, interviews, and meetings are synchronous, same-place mechanisms. These mechanisms are characterized by a low content of technical information. However, same-place mechanisms allow for the transfer of nonverbal information that is critical during consensus building and negotiation. Also, same-place mechanisms allow for clarifications and, more generally, a higher degree of flexibility. The cost of synchronous mechanisms is high, especially when many participants are involved. Synchronous different-place groupware attempts to reduce the cost of synchronous communication by not requiring all participants to be at the same location. Also, groupware promises to make a variety of information available (e.g., running code, editable documents, hyperlink structures), thus increasing the communication bandwidth among participants. Synchronous mechanisms of communication are listed in Table 3-3 and described in the following sections.

Questionnaires, fax, E-mail, newsgroups, and different-time groupware are asynchronous mechanisms of communication. They allow a large number of details to be communicated quickly. Moreover, they allow for a much larger number of receivers, which makes these mechanisms ideal for document and code releases. Asynchronous communication has the disadvantage of making nonverbal information inaccessible, thus increasing the potential for misunderstandings. Asynchronous mechanisms of communication are described in Table 3-4 and described in Sections 3.4.6–3.4.9.

Table 3-3 Synchronous mechanisms of communication

Mechanism	Supported modes
Hallway conversations (Section 3.4.1)	Request for clarification Request for change
Questionnaires and structured interviews (Section 3.4.2)	Application domain knowledge elicitation Postmortem review
Meetings (face-to-face, telephone, video) (Section 3.4.3)	Client review Project review Inspection Status review Postmortem review Brainstorm Issue resolution
Same-time, different-place groupware (Section 3.4.5)	Client review Project review Inspection Brainstorm Issue resolution

Table 3-4 Asynchronous mechanisms of communication

Mechanism	Supported modes
Electronic mail (Section 3.4.6)	Release Change request Brainstorm
Newsgroups (Section 3.4.7)	Release Change request Brainstorm
World Wide Web (Section 3.4.8)	Release Asynchronous code inspections Change request Brainstorm
Lotus Notes (Section 3.4.9)	Release Change request Brainstorm

3.4.1 Hall way con ver sations

Hallway con versations are event-driven, informal exchanges of information based on opportunity. Two participants meet by accident and take advantage of the situation to exchange information.

Example. Two project participants, Sally and Bob, meet at the coffee machine. Sally, member of the user interface team, remembers that Bob is a member of the notification team, which is responsible for the communication between the client subsystems and the server. All morning Sally has been experiencing random failures when receiving packets from the server. She is not sure if the problem comes from the server, the communication subsystem, or her code. Bob answers that he was not aware that the server was being used at this time and that he had been testing a new revision of the communication system, explaining the behavior that Sally had observed. Bob had bypassed configuration management policy to save time.

Hallway conversations represent a substantial part in the overall project communication. They are cheap and effective for resolving simple problems that are caused by a lack of coordination between project members. In addition, they are also effective in supporting the exchange of operational knowledge, such as frequently asked questions about tools, procedures, or the location of project information. The drawbacks of hallway conversations include their small audience and lack of history: Important information can be lost and misunderstandings may occur when the content of the conversation is relayed to other participants. Moreover, no document, database, or electronic message can be accessed when referring to a decision that was made and communicated during a hallway conversation.

3.4.2 Questionnaires and structured inter vie ws

The objective of a **questionnaire** is to elicit information from one or more persons in a structured manner. Questionnaires are typically used for eliciting domain knowledge from users and experts, understanding user requirements and priorities. They can also be used for extracting lessons learned during a postmortem review. Questionnaires can include both multiple choice questions and open-ended questions.

Questionnaires have the advantage of eliciting reliable information at minimal cost to the user. Questionnaires can be answered by users independently, then reviewed and analyzed by the analyst or developer. Clarifications of ambiguous or incomplete answers are then obtained during a **structur ed inter view**. The drawback of questionnaires is that they are difficult to design. However, the cost of requirements errors and misunderstandings between the client and the developer often justify their cost. Subsequently, sufficient information is gathered about the domain and a requirements analysis document is written, so that most revisions to the systems and additional issues are addressed in client reviews.

3.4.3 Meetings

Face-to-face **meetings**¹ enable a number of participants to share, review, and negotiate issues and solutions. To date, meetings are the only mechanism that allow effective resolution of issues and the building of consensus. The drawback of meetings is their cost in resources and the difficulty in managing them. In order to increase the information transfer and the number of decisions made during a meeting, roles are assigned to selected participants:

- The **primary facilitator** is responsible for organizing the meeting and guiding its execution. The facilitator writes an agenda describing the objective and the scope of the meeting. The agenda is generally released before the meeting for preview by its participants. This allows participants to decide whether the meeting is relevant to them and to allow preparation of support material for the meeting.
- The **minute taker** is responsible for recording the meeting. The minute taker may take notes on paper or on a laptop computer, organize them after the meeting, and release them shortly after the meeting for review by the meeting participants. This enables the participants to reiterate their commitment to the outcome of the meeting. The written record of the meeting makes it also easier for participants to share information with members that were not present to the meeting.
- The **time keeper** is responsible for keeping track of time and notifying the facilitator if a discussion consumes more time than is allocated by the agenda.

A **meeting agenda** consists of at least three sections: a header identifying the planned meeting location, time, and participants; a list of items participants will report on; and a list of issues that need to be discussed and resolved in the meeting. Each information-sharing and discussion item is also assigned a time that allows the timekeeper to ensure that the meeting ends on time. Figure 3-9 is an example of a meeting agenda.

A set of **meeting minutes** consists of three sections that correspond to the sections of the agenda. In addition, meeting minutes include a section describing the action items resulting from the meeting (i.e., items describing actions to be taken by the meeting participants as a consequence of the meeting). The header section contains the actual meeting location, time, and participants. The information-sharing item section contains the information that was shared during the meeting. The decision-item section contains a record of the decisions that were made (and not made). Figure 3-10 is an example of meeting minutes.

Although meetings conducted in a single location are most efficient, it is possible to conduct meetings when participants are distributed geographically by using teleconferencing or video conferencing. This reduces costs at the expense of a lower bandwidth and lower reliability. A well-structured agenda available prior to the meeting becomes crucial as floor control

1. Meeting procedures described in this section are derived from [Kayser, 1990].

<i>Header information identifying the meeting and audience</i>	When and Where Date : 1/30 Start : 4:30pm End : 5:30pm Building : Wean Hall Room : 3420	Role Primary Facilitator : Peter Timekeeper : Dave Minute Taker : Ed
<i>Desired outcome of the meeting</i>	1. Objective Resolve any requirements issues that prevent us from starting prototyping.	
<i>Action items to be reported on</i>	2. Status [Allocated Time: 15 minutes] Dave: State of command parsing code	
<i>Issues scheduled to be discussed (and resolved) during the meeting</i>	3. Discussion items [Allocated Time: 35 minutes] 3.1 How to deal with arbitrarily formatted input data sets? 3.2 How to deal with output data? 3.3 Command parsing code (modifiability, backward compatibility)	
<i>The wrap-up period is the same for all meetings</i>	4. Wrap up [Allocated Time: 5 minutes] 4.1 Review and assign new action items 4.2 Meeting critique	

Figure 3-9 An example of a meeting agenda.

becomes difficult with lower audio and visual quality. Also, knowledge of the individual voices and particularities improves communication among participants.

When writing a meeting agenda, the facilitator should be as concrete as possible without adding to the length of the agenda. It is often tempting to develop a generic template agenda and reuse it systematically without modifications. This has the drawback of taking the substance out of the meeting process, turning it into a bureaucratic procedure. Figure 3-11 is an example of contentless agenda. By only modifying the header, this agenda could apply to most subsystem meetings, and thus, does not convey any new information to the participants are not aware of.

3.4.4 Reviews

Reviews are formal meetings during which the quality of an artifact (e.g., document or code) is evaluated by an external party. They are typically conducted as formal presentations, preceded by a release, and followed by a number of requests for change. Reviews are more costly than regular meetings, given that they often require travel for the client or the developers. As with regular meetings, an agenda is posted and sent to the external party prior to the review. Minutes of the review are carefully recorded and includes requests for changes and other comments and issues that the external party may have.

<i>Header information identifying the meeting and audience</i>	When and Where Role
	Date : 1/30 Primary Facilitator : Peter
	Start : 4:30pm Timekeeper : Dave
	End : 6:00pm Minute Taker : Ed
	Building : Wean Hall Attending : Ed, Dave, Mary, Peter, Alice
	Room : 3420
<i>Verbatim from agenda</i>	1. Objective ...
<i>Summary of the information that was exchanged</i>	2. Status ...
<i>Record of issue discussion and resolution</i>	3. Discussion 3.1 Command parsing code is a 1200–1300 line if statement. This makes it fairly hard to add new commands or to modify existing commands without breaking backward compatibility with existing clients. Proposals: 1) Restructure the command parsing code by assigning one object per kind of command. 2) Pass all command arguments by name. The latter would make it easier to maintain backward compatibility. On the other hand, this would increase the size of the commands, thus increasing the size of the command file. Resolution: Restructure code for now. Revisit this issue if backward compatibility is really an issue (the calling code might be rewritten anyway). See AI[1]. ... <i>Discussion of the other issues omitted for brevity</i>
<i>Additions and modifications to the task plan</i>	4. Wrap up AI[1] For: Dave. Revisit command parsing code. Emphasis on modularity. Coordinate with Bill from the database group (who might assume backward compatibility). ... <i>Other action items and meeting critique omitted for brevity</i> ...

Figure 3-10 An example of meeting minutes.

The scope of a review (e.g., a client review) may require the review to be organized long before its execution. Flexibility should be left in the organization, however, such that the object of the review (e.g., the requirements document) can evolve as late as possible (e.g., a few days or a few hours before the review). This flexibility can enable the developers to discover more issues, points of clarification, or alternative solutions and minimize the time during which the state of the deliverable needs to be frozen.

<i>Header information identifying the meeting and audience</i>	When and Where	Role
	Date : 1/30	Primary Facilitator : Peter
	Start : 4:30pm	Timekeeper : Dave
	End : 5:30pm	Minute Taker : Ed
	Building : Wean Hall	
	Room : 3420	
<i>Desired outcome of the meeting</i>	1. Objective	
	Resolve open issues	
<i>Action items to be reported on</i>	2. Status [Allocated Time: 15 minutes]	
	Dave: Dave's action items	
<i>Issues scheduled to be discussed (and resolved) during the meeting</i>	3. Discussion items [Allocated Time: 35 minutes]	
	3.1 Requirements issues	
	3.2 Design issues	
	3.3 Implementation issues	
<i>The wrap-up period is the same for all meetings</i>	4. Wrap up [Allocated Time: 5 minutes]	
	4.1 Review and assign new action items	
	4.2 Meeting critique	

Figure 3-11 An example of poor meeting agenda.

3.4.5 Same-time, different-place groupware

Same time, different place groupware are tools that allow distributed users to collaborate synchronously. While, for a long time, these tools were only available in the realm of research [Grudin, 1988], they have been moving slowly into the commercial world with the recent popularization of Internet-based chatrooms and forums. Tools such as Teamwave [Teamwave, 1997] or Netmeeting enable a group of participants to collaborate synchronously over a shared workspace. They provide a meeting metaphor: Users “enter” a chat room, which then allows them to view a graphic or a text under consideration. All users see the same state. Usually only one can modify it at any one time. Floor control can be anarchic (whoever takes the floor has it) or sequential (whoever has the floor relinquishes it to the next user).

A weakness of same-time groupware is the difficulty in coordinating users. Typing takes more time than users are prepared to invest. Written words need to be chosen more carefully, given that nonverbal information is lost. Moreover, slight glitches in the network connection may represent enough interference for user coordination to be lost.

Same-time groupware, however, can become a useful substitute for video conferencing when combined with an audio channel: Users can view the same document and discuss it normally while scrolling and pointing to specific areas of the document. In all cases, different-place collaboration is still a nontrivial exercise that needs to be scripted and planned in advance.

Collaborative development of procedures for supporting collaboration is an challenging task when proximity and nonverbal communication are not available.

3.4.6 Electr onic mail

E-mail enables a sender to transmit an arbitrary text message to one or more recipients. The actual transmission of the message takes anywhere from a few seconds to a few hours or days. The message is received in a mailbox and kept until the receiver reads it and archives it. E-mail, in a development project, is often used in place of office memos or telephone. Participants can exchange with this mechanism a broad range of artifacts, from short informal notices to large documents. Finally, E-mail is standardized by a number of Internet Request for Comments (RFCs) that are widely implemented and used. As a consequence, E-mail is used across different organizations and platforms.

Because of its asynchronous nature and short latency, E-mail is ideal for supporting event-driven communication modes, such as requests for clarification, requests for change, and brainstorm. E-mail is also often used for announcing releases and exchanging intermediate versions of documents. Unlike the phone, E-mail enables participants to receive E-mail when they are available without introducing constraints on the sender.

A drawback of E-mail is that communication can be perceived as a trivial task. Typing a one-line message and sending it takes only a few seconds. But the same one-line message can be taken out of context and misunderstood, sent to the wrong person, or, most often, lost or unread. Because of the asynchronous nature of E-mail, potential for information loss is more likely. Project participants usually adjust to these drawbacks quickly after several serious misunderstandings have occurred.

3.4.7 Newsgr oups

Newsgr oups offer a similar metaphor to E-mail. However, instead of sending a message to a number of recipients, the sender posts a message to a newsgroup (often also referred to as a forum). Recipients subscribe to the newsgroups they want to read. Post and read accesses to a specific newsgroup can be controlled. Newsgroups follow a similar set of widespread standards as E-mail. There are many public domain and commercial tools available for managing a news server. Often, software for reading and sending E-mail is integrated with newsgroup readers, which facilitates the integration between E-mail and newsgroups.

Newsgroups are suited for notification and discussion among people who share a common interest. In a software project, teams may have a number of newsgroups; for example, in Figure 3-12, one for posting meeting agendas and minutes, another one for requests for clarification, yet another one for issues. Instead of enumerating the addresses of the team members every time a message is posted, the sender only has to name the targeted newsgroup. Typical modes that can be supported via newsgroups include requests for clarification, requests for change, releases, and brainstorm.

<i>Newsgroup for communication with client</i>	cmu.academic.15-413.client
<i>Global newsgroups read by all developers</i>	cmu.academic.15-413.announce cmu.academic.15-413.discuss cmu.academic.15-413.issues cmu.academic.15-413.equipment
<i>Team newsgroups read mostly by the members of a single team (e.g., user interface team)</i>	... cmu.academic.15-413.ui.announce cmu.academic.15-413.ui.discuss cmu.academic.15-413.ui.minutes
<i>Special purpose newsgroups</i>	... cmu.academic.15-413.integration-tests

Figure e 3-12 An example of a newsgroup structure.

3.4.8 World Wide Web

The **World Wide Web** (WWW, also referred to as the “Web”) provides the user with a hypertext metaphor. A WWW browser displays a document to the user that may contain hyperlinks to other documents. A Universal Resource Locator (URL) is associated with each link and describes to the browser the location of the target document and its method of retrieval. The WWW has growing popularity as hypertext documents were extended to contain embedded images, animations, and executable scripts.

In a software development project, the WWW is ideal for organizing and providing access to released information, such as preliminary and final versions of the deliverable documents, code, references (e.g., vendor home pages), and tool documentation.

Although the type of documents and functionality of WWW browsers evolves rapidly, the Web does not support in an equally straightforward manner the exchange of rapidly changing information such as newsgroup messages. However, with the integration of newsgroups into Netscape and other browsers, the WWW can play a significant part in a project’s communication infrastructure.

Figure 3-13 displays the home page of the OWL project [OWL, 1996]. It contains references to the most important parts of the Web site such as the latest version of the project documents and review presentations, newsgroups, team organization charts, team Web pages, and references. The project Web site is managed by a single Web master. The pages associated with each team are administered by the teams with guidance from the project Web master.

3.4.9 Lotus Notes

Lotus Notes is a commercial tool (as opposed to a set of standards, as in the case of E-mail, newsgroups, and WWW) that provides functionality comparable to that of the Web. It provides a metaphor, however, that is closer to a database management system than to an

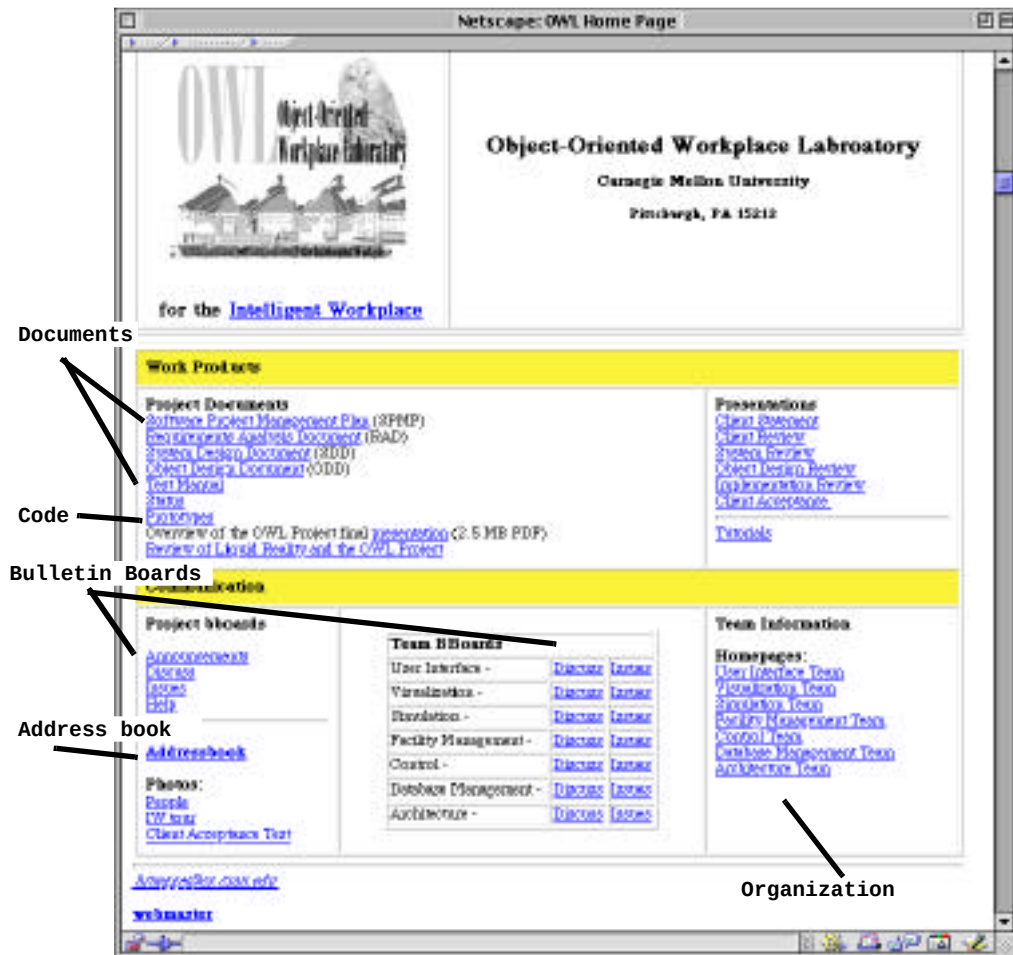


Figure e 3-13 An example of a project home page for the OWL project.

E-mail system. Each user sees the information space as a set of databases. Databases contain documents composed of a set of fields. Documents are created and modified through a set of forms. Users collaborate by creating, sharing, and modifying documents. Each database can follow different templates and support application-specific functionality. For example, one database can support discussions (see Figure 3-8), in which case its documents are messages and replies. Another database can support a change request tracking system, in which case documents can be change request forms, revision logs, or release notes.

Lotus Notes differs from the Web by addressing well two additional aspects of information management: access control and replication. Access can be controlled at the level of a database (e.g., deny developers access to a management database), a document (e.g., disallow a developer to modify a change request posted by a user), or a field (e.g., allow only an administrator to modify the author field of a document). Access control can also be set in terms of groups of users. The concept of groups enables the administrator to express access control in terms of the organization of the project rather than of its individuals.

The main different between Lotus Notes and the Web has been that Notes is a proprietary standard: Both client and server programs need to be purchased from IBM. The Web, on the other hand, is so ubiquitous because it relies on a set of public standards. It is possible to construct a Web application, accessible by users with different browsers, entirely out of public domain software (at the cost of reliability, support, and some functionality). In order to hedge the competition by the Web and to take advantage of the widespread use of the Web browsers, Lotus Notes introduced Domino, a product enabling Web browsers to access Lotus Notes databases through a standard browser.

Example. Each team may have a database for team announcements. Documents in the announcement databases are either announcements or replies. Only team members can post announcements, whereas anybody can read announcements and post replies. The Lotus Notes administrator elected to organize project participants into access groups. Each access group corresponds to a team. Furthermore, the administrator set up the access of each announce database such that only members of the team who owns the database may post announcements. Assume Sally, member of the user interface team, is reassigned to the notification team. The administrator needs only to remove Sally from the user interface access group and add her to the notification access group to reflect the organizational change. In the event the administrator did not organize access by group, he would have had to review and change the access to all the databases to which Sally had access.

3.4.10 Discussion

Project communication is pervasive. It occurs at many levels under different forms and can be supported by different tools. Communication is also a fundamental skill for project participants. Not all participants may be familiar with the latest development technology used by the project, but they all should have some idea of how information is disseminated through the project. For example, participants who have been using E-mail every day for previous projects may be reluctant to switch to an alternate infrastructure, such as newsgroups or Lotus Notes. For these reasons, the communication infrastructure, including the collection of tools used to support communication, needs to be carefully designed and put in place before the start of the project. The following criteria can be used in order to facilitate this:

- *Completeness.* Are all modes present in the project supported by one or more tools?
- *Complexity.* Are too many tools used to support communication? Can the information be duplicated or fragmented to the point that it is hard to find?

- *Reliability*. Is the primary communication tool reliable? What is the impact on the project if it fails?
- *Maintenance*. Does the infrastructure need a full-time administrator? Are the resources available for such a role? Are the skills available for such a role?
- *Ease of transition*. Is the metaphor presented by the infrastructure familiar enough to the users?

Transition is a critical step in setting up the communication infrastructure, even more than for other tools: The infrastructure will be used only if a majority of participants use it effectively. Otherwise, communication will occur through alternate ad hoc channels, such as hallway conversation, and the dissemination of information will be inconsistent and unreliable. In the next section, we provide an example of project communication infrastructure and of its transition.

3.5 Project communication activities

In this section, we examine the communication needs of an example project, select communication modes and mechanisms to support them, and describe their integration and their transition of the infrastructure into the project.

3.5.1 Identifying communication needs

This example project is a greenfield engineering effort with all new developers. The first phases of problem definition and preliminary design are handled directly by management in face-to-face meetings. Developers have little or no direct access to the client during these phases. Teams are also not expected to conduct inspections or walkthroughs until their level of expertise has sufficiently matured.

The preliminary design enables the management to define the initial decomposition of the system into subsystems (see Chapter 11, *Project Management*). Each subsystem is assigned to a team of developers. Additionally, cross-functional teams (e.g., documentation team, integration team) are formed to support the subsystem teams. Finally, each team has liaisons to the cross-functional teams to facilitate information transfer among teams. The initial project organization is displayed in Figure 3-14.

When defining subsystems, management attempts to identify groups of functionality that can be easily decoupled in order to reduce dependencies across subsystem teams. As a consequence, most asynchronous communication is expected to occur within teams and liaisons, whereas project-level communication is expected to happen at well-defined scheduled events, such as, project reviews and releases. A priori, management views the following modes as critical for cross-team communication:

- Client review—quarterly
- Project review—monthly

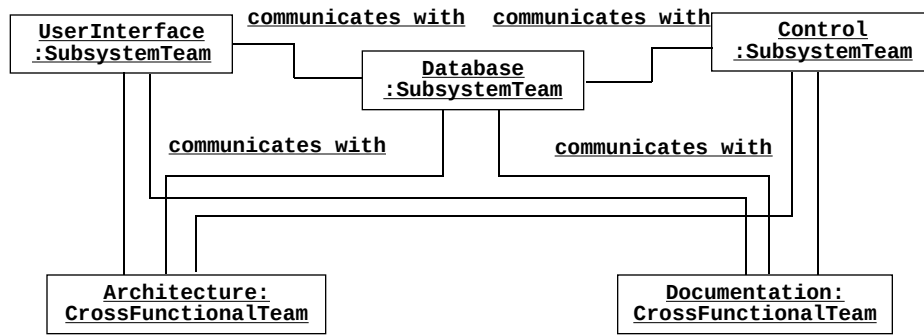


Figure e 3-14 An example of a project organization, used as the basis for the communication infrastructure (UML object diagram). Associations represent communication channels via liaisons.

- Releases—weekly
- Requests for clarification—as needed
- Requests for change—as needed
- Project issue resolution—as needed

Similarly, management views the following modes as critical for intrateam communication:

- Status review—weekly
- Brainstorming—weekly
- Team issue resolution—as needed
- Request for clarification—as needed
- Request for change—as needed

Given the increasing importance of reviews in management's perspective of communication, management decides to select formal meetings for conducting client and project reviews until more expertise in the use of synchronous groupware tools is acquired. Team status and brainstorming is handled in a weekly team meeting. Management selects a forum-based groupware tool for supporting releases and the bulk of the asynchronous communication. Finally, management expects informal hallway conversations and liaisons to handle communication during crises, in case the formal communication channels break down.

3.5.2 Setting up an infrastructure

Two sets of forums are created to support project and team communication, respectively. Members subscribe to all project forums and to their team's forums. Project forums include:

- *Announce.* Major events (e.g., review agendas, releases) are announced by management by posting to this forum. Only management can post announcements to this forum; project members can post replies and read all documents.
- *Discuss.* Project-level requests for clarifications and requests for change are posted in this forum. Discussion about the requests (e.g., arguments and alternate solutions) are posted as replies to the original messages. All project members can post to this forum and read its documents.
- *Issues.* Open issues and their current state are posted in this forum. All project members can post to this forum and read its documents.
- *Documents.* The latest versions of the project deliverables (e.g., Requirements Analysis Document, System Design Document) and other internal project documents (e.g., Software Project Management Plan) are posted in this forum. Only the documentation team can post documents to this forum. All project members can post replies (i.e., annotations to the documents) and read the documents.
- *Equipment list.* This forum contains descriptions of available equipment and its status (e.g., availability, current borrower). Only the equipment manager can post to this forum.

The team forums are similar to the project forums, except that they support team communication. Team forums include:

- *team discussion*
- *team issues*
- *team documents*

Each project member may read any other team's forum. Team members can post only to their own team's forums. Note that the forums can be created as soon as the subsystem decomposition is relatively stable. Once forums and group accesses are set up, accounts for individual members can be created as the project is staffed.

3.5.3 Organizing client and project reviews

Client reviews are conducted after the release of the requirements analysis document and after the delivery of the system. Project reviews are conducted to review the system design documents, the detailed object design, and the test. A project review may also be conducted before delivery as a dry run for the client acceptance test.

The project management decides to schedule all reviews during the planning phase (see Table 3-5).

Table 3-5 An example of a review schedule

Review	Date	Deliverable (release due 1 week before review)
Client review	week 7	Requirements Analysis Document
System design review	week 9	System Design Document
Object design review	week 13 (2 sessions)	Object Design Document
Internal review	week 16	Unit and integration tests
Client acceptance test dry run	week 17	All project deliverables
Client acceptance test	week 17	All project deliverables

The management also introduces procedures for organizing reviews:

1. The deliverables being reviewed are released 1 week² prior to the review.
2. Shortly after the release, the management publishes a draft agenda listing presentation topics for each team. The initial draft of the agenda is posted as a Lotus Notes document in the project *Announce* forum.
3. Candidate presenters replies to the original agendas and refine the presentation topic. The management modifies the agenda based on the replies.
4. Presenters submit their slides by replying to the agenda and including the slides in the reply. The management collates the slides before the presentation and updates the agenda.

The management also assigns the responsibility of minute taker, using the same procedure, to a project member, who will be briefed on how to take minutes by the management. During the review, the minute taker is using a laptop and carefully records all the questions from the audience and the answers. Finally, within a day of the review, the minute taker and management merge their notes and generate a list of action items to be completed as a result of the review and a list of open issues that could not be resolved during the review. The postprocessed minutes are posted on the *Announce* forum.

The emphasis on using the communication infrastructure for coordinating the organization of the review and the submission of slides enables more information to be captured, and thus, accessible in the information space of the project.

2. This leaves slack time for late documents. Realistically, some deliverables are often delivered as late as 1 day before the review. The critical issue here is: 1) Can the deliverable be made available to all review participants and 2) Do they have enough time to review them?

3.5.4 Organizing weekly team meetings

Management decides to institute a weekly team meeting for all teams in order to support status reviews, brainstorming, and issue resolution. The weekly team meeting is organized and captured as described in Section 3.4.3. Given that the example project is a first-time development with a flat staffing, few members know each other socially and technically. Moreover, few of them are familiar with formal meeting roles and procedures. Management takes the opportunity of the first weekly team meeting to introduce meeting procedures, explain the importance of these procedures, and train team members in their use. Figure 3-15 displays the agenda posted by the management for the first meeting.

The goal of the first meeting is to train participants by example. Discussion about the procedures is encouraged. The meeting and group roles are explained to the participants and assigned by the team for the rest of the project. The role of facilitator is emphasized in that its purpose is to increase the efficiency of the meeting, not to impose decisions. Team members are taught that any meeting participant can take the role of secondary facilitator; that is, any participant can intervene in the discussion in order to put the meeting back within the scope of the agenda. Participants are taught keyword phrases for standard situations. (e.g., *Let me play the role of secondary facilitator* stands for “The scope of the current discussion is outside the agenda. Let us get back on track”; *Can we pop up a level?* stands for “The discussion has delved into a level of detail that is unnecessary for this audience. Actually, most of us have already gotten lost.”) More generally, team members are taught that it is easy to waste time during a meeting, and that the primary goal of any meeting is to communicate efficiently and accurately such that they can go back to their respective tasks.

Management decides to have roles be rotated on a regular basis so that all participants have the opportunity to fill every role. This has the advantage of creating redundant skills across participants and better sharing of information. The drawback is that, in the short term, participants will not have time to mature into their role and thus become highly effective at a given task. Requiring early role assignment, role rotation, and more generally, meeting procedures to be in place early may introduce turbulences at the beginning of the project but represents a healthy investment in the long term. Management takes the position that everyday meeting and communication skills should be in place well before crisis-driven communication needs surface during the implementation and code activities.

The teams are responsible for assigning meeting roles and posting them in their respective team *Announce* forum. The meeting facilitator is required to post the initial draft of the agenda, composed of action items taken from the previous meeting’s minutes and issues taken from the *Issues* forum.

In the team *Announce* forum 1 day before the status meeting: The minute taker is required to post the minutes within a day from the meeting, as a reply to the corresponding agenda. Other team members may comment on the agenda and minutes by posting replies. The facilitator or the minute taker may then amend the corresponding document.

When and Where	Role
Date : 1/9	Primary Facilitator : Alice
Start : 4:30pm	Timekeeper : Dave
End : 5:30pm	Minute Taker : Ed
Building : Wean Hall	
Room : 3420	
1.Objective	
Become familiar with project management roles for a medium-scale project with a two-level hierarchy. In particular:	
• Understand the difference between a role and a person • Group roles are assigned to people • Meeting times are finalized • First set of action items for next meeting	
2.Status and information sharing [Allocated time: 40 minutes]	
2.1. How to organize a meeting	
Meeting ground rules	
• Active listening • Active participating • Punctual attendance • No one-on-one or side meetings • Respect the agenda • Keep time • Willingness to reach consensus • Freedom to check process and ground rules	
Meeting Roles	
• Primary Facilitator • Timekeeper • Minute taker • Scribe	
2.2 Following the Agenda	
<i>Omitted for brevity</i>	
3.Discussion items [Allocated time: 15 minutes]	
3.1 Team address book	
3.2 Meeting roles assignments	
3.3 Group roles assignment	
4. Wrap up [Allocated time: 5 minutes]	
4.1 Review and assign new action items	
4.2 Meeting critique	

Figure 3-15 First weekly team meeting agenda.

Meeting roles and procedures are often perceived as overhead. Management is aware of that perception and invests time in the beginning of the project to illustrate the benefits of the meeting procedures. In the first weeks of the project, management systematically reviews the agendas and minutes of the first few weekly meetings, suggests time-saving improvements to the facilitators (e.g., keeping an active document containing open issues and active action items from which the agenda can be cut and pasted), and to the minute takers (e.g., focusing on capturing the action items and unresolved issues first, then focusing on the discussion).

3.5.5 Transition issues revisited

Learning to use a new communication infrastructure in a project can be difficult [Orlikowski, 1992]. As mentioned earlier, it is critical that the transition of the communication infrastructure occur early in the project. On the one hand, once a critical mass of information and users is attained, the other users and more information will make its way into the communication infrastructure. On the other hand, if critical mass is not attained early, alternate informal channels will be set in place and solidify, making it difficult to correct them later.

Management can encourage the transition by making available a maximum of structured information up front (e.g., the problem statement, the Software Management Plan, agenda and document templates), require project participants to accomplish simple tasks using the infrastructure (e.g., posting their team roles in their *Announce* forum), and providing feedback using the communication infrastructure.

3.6 Exercises

1. You are a member of the user interface team. You are responsible for designing and implementing forms collecting information about users of the system (e.g., first name, last name, address, E-mail address, level of expertise). The information you are collecting is stored in the database and used by the reporting subsystem. You are not sure which fields are required information and which are optional.

How do you find out?

2. You have been reassigned from the user interface team to the database team due to staff shortages and replanning. The implementation phase is well underway.

In which role would you be most proficient given your knowledge of the user interface design and implementation?

3. Assume the development environment is Unix workstations, and the documentation team uses the Macintosh platform for writing documentation. The client requires the documents to be available on Windows platforms. Developers produce the design documentation using FrameMaker. The documentation team uses Microsoft Word for the user-level documentation. The client submits corrections on hardcopies and does not need to modify the delivered documents.

How could the information flow between the developers, the technical writers, and the client be set up (e.g., format, tools, etc.) such that duplication of files is minimized while everybody's tool preferences and platform requirements are still satisfied?

4. Which changes in the organization and communication infrastructure would you recommend for a successor of the Ariane 5 project as a consequence of the Ariane 501 failure described at the beginning of this chapter?

References

- [DIAMOND, 1995] *DIAMOND Project Documentation*, School of Computer Science, Carnegie Mellon Univ., Pittsburgh, PA, 1995.
- [Dutoit & Bruegge, 1998] A. H. Dutoit & B. Bruegge, "Communication metrics for software development," *IEEE Transactions on Software Engineering*, Aug. 1998.
- [Fagan, 1976] M. E. Fagan, "Design and code inspections to reduce errors in program development," *IBM System Journal*, vol. 15, no. 3, pp. 182–211, 1976.
- [FRIEND, 1994] *FRIEND Project Documentation*, School of Computer Science, Carnegie Mellon Univ., Pittsburgh, PA, 1994.
- [Grudin, 1988] J. Grudin, "Why CSCW applications fail: Problems in design and evaluation of organization interfaces," *Proceedings CSCW'88*, Portland, OR, 1988.
- [Kayser, 1990] T. A. Kayser, *Mining Group Gold*. Serif, El Segundo, CA, 1990.
- [Lions, 1996] J.-L. Lions, *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*, <http://www.esrin.esa.it/htdocs/tidc/Press/Press96/ariane5rep.html>, 1996.
- [Orlikowski, 1992] W. J. Orlikowski, "Learning from Notes: organizational issues in groupware implementation," in *Conference on Computer-Supported Cooperative Work Proceedings*, Toronto, Canada, 1992.
- [OWL, 1996] *OWL Project Documentation*, School of Computer Science, Carnegie Mellon Univ., Pittsburgh, PA, 1996.
- [Reeves & Shipman, 1992] B. Reeves & F. Shipman, "Supporting communication between designers with artifact-centered evolving information spaces," in *Conference on Computer-Supported Cooperative Work Proceedings*, Toronto, Canada, 1992.
- [Saeki, 1995] M. Saeki, "Communication, collaboration, and cooperation in software development—How should we support group work in software development?" in *Asia-Pacific Software Engineering Conference Proceedings*, Brisbane, Australia, 1995.
- [Seaman & Basili, 1997] C. B. Seaman & V. R. Basili, "An empirical study of communication in code inspections," in *Proceedings of the 19th International Conference on Software Engineering*, Boston, MA, 1997.
- [Subrahmanian et al., 1997] E. Subrahmanian, Y. Reich, S. L. Konda, A. Dutoit, D. Cunningham, R. Patrick, M. Thomas, & A. W. Westerberg, "The n -dim approach to building design support systems," *Proceedings of ASME Design Theory and Methodology DTM'97* ASME, New York, 1997.
- [Teamwave, 1997] Teamwave Inc., <http://www.teamwave.com>, 1997.